

Contents

guide 01_prefix tuning	2
0. 这篇论文一句话	2
1. 为什么 2021 年需要这篇论文	2
2. 原论文结构地图	3
3. 核心概念	4
3.1 Full fine-tuning	4
3.2 Prompt	4
3.3 Continuous prompt	5
3.4 Prefix	5
3.5 Key/value prefix	5
3.6 Reparameterization	6
3.7 Extrapolation	6
4. Figure 1: full fine-tuning vs prefix-tuning	6
5. Problem statement: conditional generation	7
6. Prefix-Tuning 的正式方法	7
6.1 Autoregressive LM	7
6.2 Encoder-decoder LM	8
6.3 Reparameterization	8
7. 张量形状	9
7.1 存储量直觉	9
8. 为什么不是只学输入 embedding	10
9. 为什么 prefix 放在最前面	10
10. 训练目标	11
11. 本地代码映射	12
12. 最小代码片段	13
13. 实验一: Table-to-text 主结果	14
14. 实验二: Summarization	14
15. 实验三: Low-data setting	15
16. 实验四: Extrapolation	16
17. 实验五: Prefix length	16
18. 实验六: Embedding-only and infix ablation	17
19. 实验七: Initialization	17
20. Discussion: 个性化和批处理	17
20.1 Personalization	18
20.2 Batching across users	18
20.3 Inductive bias	18
21. 与后续 PEFT 方法的关系	18
22. 常见误解	19
23. 30 到 60 分钟本地实验	19
24. 用 AI agent 正确学习这篇	20
25. 局限性	21
26. 现代意义	22
27. 闭卷掌握检查	22
28. 最小复述模板	23

guide_01_prefix_tuning

原论文: Prefix-Tuning: Optimizing Continuous Prompts for Generation

本地原文 PDF: [learning/prompt-tuning-family/paper/01_prefix_tuning.pdf](#)

作者: Xiang Lisa Li, Percy Liang

机构: Stanford University

arXiv: 2101.00190

年份: 2021

类型: parameter-efficient tuning, continuous prompt, natural language generation

0. 这篇论文一句话

Prefix-Tuning 的核心想法是:

不要为每个生成任务复制一整份 GPT-2 或 BART。冻结原来的语言模型参数, 只训练一小段 task-specific continuous prefix, 并让 Transformer 后续 token 像注意真实上下文一样注意这段 prefix。

它不是普通 prompt engineering。普通 prompt 是离散文字。Prompt tuning 只改输入 embedding。Prefix-Tuning 更激进:

它在每一层 Transformer attention 的 key/value 侧放入可训练 prefix activation。

这让 prefix 不是只在第 0 层输入端产生影响, 而是直接参与每一层注意力计算。论文证明, 在 table-to-text 和 summarization 生成任务上, 这可以用极少任务参数接近甚至超过 full fine-tuning, 尤其在 low-data 和 extrapolation 设置里更稳。

1. 为什么 2021 年需要这篇论文

把时间拉回 2020 到 2021 年。

当时大语言模型已经很有用, 但主流下游使用方式仍然是 full fine-tuning:

- 为每个任务加载一个 pretrained LM。
- 用该任务数据更新所有参数。
- 部署时为每个任务保存一整份模型副本。

如果只有一个任务, 这很自然。但如果有一百个任务、一千个用户、或者许多垂直场景, full fine-tuning 的存储和维护会很快失控。

论文开篇把这个痛点说得很明确:

- GPT-2 已经有数亿参数。
- GPT-3 已经到 175B 参数量级。
- 每个任务保存一份完整模型, 在工程上非常昂贵。

当时已有几类替代方案:

1. Adapter-tuning

冻结大部分 LM 参数, 在 Transformer 层之间插入小模块。效果不错, 但仍然需要在网络内部增加 task-specific layer, 常见额外参数是几个百分点。

2. In-context learning / prompting

GPT-3 展示了用自然语言 instruction 和 examples 让模型完成任务。优点是不用训练。问题是上下文窗口有限，无法充分利用大量训练数据；并且手写 prompt 不稳定。

3. Discrete prompt search

自动寻找触发词或模板。问题是离散 token 搜索困难，而且真实词表限制了表达能力。

Prefix-Tuning 的位置就在这里：

它保留 prompting 的直觉，但把 prompt 从文字变成连续向量；它保留 frozen LM 的部署优点，但比只改输入层的 soft prompt 更强；它保留 adapter 的参数高效目标，但尽量不改 Transformer 内部结构。

2. 原论文结构地图

读原文时建议这样走：

第 1 节 Introduction:

- 说明 full fine-tuning 的存储成本。
- 说明 prefix 是一段 continuous task-specific vector。
- 强调 prefix-tuning 的模块化和可跨任务部署。
- 提出主要结论: 0.1% 参数量接近 full fine-tuning，低数据和外推更好。

第 2 节 Related Work:

- Fine-tuning for generation。
- Lightweight fine-tuning。
- Prompting。
- Controllable generation。

这节的用处不是背引用，而是看作者怎样把 prefix-tuning 放在几条路线之间。

第 3 节 Problem Statement:

- 定义 conditional generation。
- 说明 autoregressive LM 和 encoder-decoder LM 的 notation。
- 写出 full fine-tuning 的 log likelihood objective。

第 4 节 Prefix-Tuning:

- 第 4.1 节给 intuition。
- 第 4.2 节给正式方法。
- 第 4.3 节解释 reparameterization。

这是最重要的章节。真正读懂本篇，关键是能把 Figure 2 和 Equation 3 讲清楚。

第 5 节 Experimental Setup:

- Table-to-text: E2E, WebNLG, DART。
- Summarization: XSUM。
- Baselines: fine-tuning, top-layer fine-tuning, adapter-tuning。
- 模型: GPT-2 medium/large, BART large。

第 6 节 Main Results:

- Table 1: table-to-text 主结果。
- Table 2: summarization 主结果。
- Figure 3: low-data。
- Table 3: extrapolation。

第 7 节 Intrinsic Evaluation:

- Figure 4: prefix length.
- Table 4: embedding-only 和 infix 消融。
- Figure 5: initialization.

这一节决定你是否真的理解 prefix 为什么设计成这样。

第 8 节 Discussion:

- Personalization.
- Batching across users.
- Prefix-tuning 的 inductive bias.
- 与 adapter-tuning 的关系。

第 9 节 Conclusion:

- 总结: 1000 倍更少任务参数, 在 full data 接近 fine-tuning, 在 low-data 和 extrapolation 更好。

附录:

- Hyperparameters.
- 更多 low-data 曲线。
- 更多 initialization 结果。
- WebNLG qualitative examples.

3. 核心概念

3.1 Full fine-tuning

Full fine-tuning 更新 pretrained LM 的所有参数。

如果原模型参数是 ϕ_{lm} , 任务数据是 (x, y) , 那么训练后得到一份新的 ϕ_{task} 。

部署多个任务时, 存储成本约是:

```
task_1: full LM copy
task_2: full LM copy
task_3: full LM copy
...
```

优点:

- 表达能力强。
- 优化目标直接。
- 对大多数任务有效。

缺点:

- 每任务存一整份模型。
- 多用户场景难以隔离又高效。
- 小数据时容易过拟合。
- 模型版本管理很麻烦。

3.2 Prompt

Prompt 是输入前面的一段上下文。

离散 prompt 是真实文字，例如：

Summarize the following article:

它的问题：

- 真实词表限制表达。
- 人工设计不稳定。
- 离散搜索很难。
- 上下文窗口会被 prompt 占用。

3.3 Continuous prompt

Continuous prompt 不要求 prefix 是真实词。它直接学习向量。

你可以把它理解成：

不是找一个词表里的 token。

而是在 embedding/activation 空间里找一段更适合任务的坐标。

这比离散 prompt 更灵活，但也更难解释。

3.4 Prefix

本文的 prefix 是一段 task-specific continuous vector。

关键点：

- LM 参数冻结。
- 每个任务训练自己的 prefix。
- 后续 token 可以 attend to prefix。
- prefix 像 virtual tokens，但不是词表 token。

论文的 prefix 不只是输入 embedding。它更接近每层 attention 的 past key/value。

3.5 Key/value prefix

Transformer self-attention 里，每个 query 会对 key 计算相似度，再用权重混合 value。

如果原始 attention 是：

Q from current token

K from previous tokens

V from previous tokens

$$\text{attention} = \text{softmax}(Q K^T / \sqrt{d_{\text{head}}}) V$$

Prefix-Tuning 做的是：

$K_{\text{all}} = \text{concat}(\text{prefix}_K, \text{normal}_K)$

$V_{\text{all}} = \text{concat}(\text{prefix}_V, \text{normal}_V)$

$$\text{attention} = \text{softmax}(Q K_{\text{all}}^T / \sqrt{d_{\text{head}}}) V_{\text{all}}$$

所以 prefix 不是在输出端加一个 bias，而是改变了每一层的注意力上下文。

3.6 Reparameterization

论文发现直接优化完整 prefix activation 不稳定。

因此训练时不直接学最终 prefix，而是学一个低维矩阵，再用 MLP 映射到每层 key/value prefix。

训练结束后，MLP 可以丢弃，只保存最终 prefix activation。

这点很重要：

- 训练时可以用较大的 MLP 帮助优化。
- 推理和存储时只保留小 prefix。

3.7 Extrapolation

论文里的 extrapolation 不是普通测试集。它故意让测试主题和训练主题不同。

例子：

- WebNLG 训练用 seen DBpedia categories，测试 unseen categories。
- XSUM 训练 news，测试 sports；或训练部分 news 子域，测试其他 news 子域。

这用于判断：

模型是否只是记住训练主题，还是能保留 pretrained LM 的泛化能力。

4. Figure 1: full fine-tuning vs prefix-tuning

论文 Figure 1 画的是两种适配方式。

可以用这个 ASCII 图复述：

Full fine-tuning

```
task data
  |
  v
+-----+
| pretrained Transformer LM |
| all weights become trainable|
+-----+
  |
  v
save a full task-specific LM copy
```

Prefix-tuning

```
task data
  |
  v
train prefix only
  |
  v
+-----+
```

```

| frozen Transformer LM      |
| same copy for all tasks    |
+-----+
      ^
      |
small task-specific prefix

```

所以这篇论文的工程意义是:

同一个 frozen LM 可以服务很多任务, 每个任务只需要换一段 prefix。

5. Problem statement: conditional generation

论文研究的是 conditional natural language generation。

每个训练样本是:

```

x = input context
y = output text

```

例子一: table-to-text

```

x = linearized table
y = natural language description

```

例子二: summarization

```

x = article
y = summary

```

模型需要学:

$p(y \text{ given } x)$

对 autoregressive LM, 可以把输入和输出串起来:

$z = [x; y]$

每个位置的 hidden activation 由当前 token 和左侧历史决定:

$h_i = \text{LM_phi}(z_i, h_{\text{before}_i})$

生成目标是最大化目标 token 的 log likelihood:

```

maximize over phi:
    sum over output positions i:
        log p_phi(z_i given h_before_i)

```

Full fine-tuning 更新的是 phi。

Prefix-Tuning 保持 phi 不动, 只更新 prefix 参数。

6. Prefix-Tuning 的正式方法

6.1 Autoregressive LM

对 GPT-2 这类 autoregressive LM, prefix 放在输入前面:

$z = [\text{PREFIX}; x; y]$

对于 prefix 位置, hidden activation 不是 LM 算出来的, 而是来自可训练矩阵:

```
if i is a prefix position:
    h_i = P_theta[i]
else:
    h_i = LM_phi(z_i, h_before_i)
```

这里:

- ϕ 是 frozen LM 参数。
- θ 是 prefix 参数。
- P_θ 是可训练 prefix activation。

虽然非 prefix token 的 h_i 由 LM 计算, 但它仍然依赖 prefix, 因为 prefix 在 causal left context 里。

这句话是理解论文的关键:

```
prefix activation is in the left context,
so every later token can attend to it.
```

6.2 Encoder-decoder LM

对 BART 这类 encoder-decoder 模型, 论文给 encoder 和 decoder 都加 prefix。

可以抽象成:

```
encoder side:
    [PREFIX_enc; x]
```

```
decoder side:
    [PREFIX_dec; y]
```

直觉:

- encoder prefix 影响模型从输入文章或表格里抽取什么。
- decoder prefix 影响生成时每一步如何组织语言。

6.3 Reparameterization

论文不是直接训练最终的 P_θ 。

它训练一个低维 prefix P_{low} , 再通过 MLP 得到最终 prefix:

```
 $P_\theta[i] = \text{MLP}_\theta(P_{low}[i])$ 
```

训练结束后:

```
keep  $P_\theta$ 
drop  $\text{MLP}_\theta$ 
```

为什么这样做?

论文说直接优化完整 prefix 对 learning rate 和 initialization 很敏感, 会不稳定并带来性能下降。MLP reparameterization 给优化过程加入归纳偏置, 让同一 prefix 位置在多层间产生结构化相关性。

7. 张量形状

假设:

L = Transformer layers
d = hidden dimension
H = attention heads
d_head = d / H
p = prefix length
B = batch size
n = input token length

推理时保存的 prefix key/value 可以看成:

prefix_kv:
shape = (L, 2, H, p, d_head)

dimension meaning:

L: layer index
2: key and value
H: attention head
p: prefix positions
d_head: per-head dimension

对某一层 self-attention:

normal K: (B, H, n, d_head)
normal V: (B, H, n, d_head)
prefix K: (B, H, p, d_head)
prefix V: (B, H, p, d_head)

concat K: (B, H, p+n, d_head)
concat V: (B, H, p+n, d_head)
query Q: (B, H, n, d_head)

attention out: (B, H, n, d_head)

注意输出长度还是 n。Prefix-Tuning 不要求模型为 prefix 位置生成 logits。prefix 是 attention memory, 不是要被预测的目标文本。

7.1 存储量直觉

对 GPT-2 medium 量级的配置:

L = 24
d = 1024
H = 16
p = 5

stored prefix params = L * 2 * p * d
= 24 * 2 * 5 * 1024
= 245760

这和保存一份数亿参数模型相比, 是完全不同的存储级别。

如果有 100 个任务:

```
full fine-tuning:
    100 full model copies
```

```
prefix-tuning:
    1 frozen model copy
    100 small prefixes
```

这就是论文一直强调 modular and space-efficient 的原因。

8. 为什么不是只学输入 embedding

很多人读到这里会问:

既然 prefix 像 virtual tokens, 为什么不只在输入层加一段 trainable embedding?

论文第 7.2 节专门做了 embedding-only ablation。

embedding-only 的结构:

```
[P_embed; x; y]
|
v
Transformer layer 1
|
v
Transformer layer 2
|
v
...

```

它只在输入层改变虚拟 token 的 embedding。上层 activation 都要靠 Transformer 自己传播。

Prefix-Tuning 的结构:

```
layer 1 attention sees prefix K/V
layer 2 attention sees prefix K/V
layer 3 attention sees prefix K/V
...
layer L attention sees prefix K/V
```

所以 prefix-tuning 对每一层都有直接控制。

论文 Table 4 显示, embedding-only 在 E2E 上明显弱于 full prefix-tuning。例如 prefix 的 BLEU 约 69.7, 而 embedding-only 即使长度到 10 或 20, BLEU 也只有 62 左右。

这说明:

输入层 soft prompt 不一定有足够表达力, 尤其对 GPT-2 medium 这类当时的模型规模和生成任务而言。

9. 为什么 prefix 放在最前面

论文还比较了 prefixing 和 infixing。

Prefixing:

[PREFIX; x; y]

Infixing:

[x; INFIX; y]

在 causal LM 里，每个位置只能看左侧。

所以:

prefixing:

x can see prefix

y can see prefix and x

infixing:

x cannot see infix

y can see x and infix

这意味着 infix 只能影响输出生成，不能影响 input x 的 activation。

Table 4 显示 infix-tuning 稍弱于 prefix-tuning。作者解释为:

prefix 放在最前面，可以影响 x 和 y; infix 放在中间，只能影响 y。

这个消融帮助我们理解 Figure 2:

Prefix-Tuning 不只是给 decoder 一点提示，它也在改变模型如何表征输入。

10. 训练目标

训练目标和 fine-tuning 的语言模型目标一样，区别只是 trainable parameters 不同。

Full fine-tuning:

trainable:

phi_lm

objective:

minimize negative log likelihood of y given x

Prefix-Tuning:

trainable:

theta_prefix only

frozen:

phi_lm

objective:

minimize negative log likelihood of y given x

更具体地说:

loss(theta) =

- sum over examples

sum over output tokens t

log p_phi_frozen(y_t given prefix_theta, x, y_before_t)

梯度路径:

```
loss
|
v
logits
|
v
frozen Transformer computation
|
v
attention uses prefix K/V
|
v
prefix parameters theta receive gradients
```

LM 参数参与前向和反向的链式计算, 但不被 optimizer 更新。

11. 本地代码映射

本仓库这篇专题有两类代码。

第一类是接近真实 HuggingFace/PEFT 的实现:

```
learning/prompt-tuning-family/src/prefix_tuning_minimal.py
learning/prompt-tuning-family/src/prefix_tuning_peft.py
learning/prompt-tuning-family/src/tests/test_prefix_consistency.py
```

它们展示:

- 如何冻结 GPT-2。
- 如何构造每层 past_key_values。
- 如何用 MLP reparameterization。
- 如何和 PEFT 实现做弱一致性检查。

第二类是我为论文导读补的无依赖玩具实现:

```
learning/prompt-tuning-family/src/prefix_tuning_original_minimal.py
learning/prompt-tuning-family/src/tests/test_prefix_original_minimal.py
```

它不下载 GPT-2, 只复现原论文机制:

- prefix_kv_shape: 每层 K/V prefix 的形状。
- saved_prefix_parameters: 推理保存参数量。
- training_reparam_parameters: 训练时 MLP 参数量直觉。
- build_layout: prefix/infix/embedding-only 的序列布局。
- causal_reach: causal attention 下每个位置能看到什么。
- attention_weights: 一个 prefix key 改变注意力权重的小例子。

建议先跑这个文件。本机请优先使用仓库 .venv, 因为系统默认 python 可能指向 Anaconda 3.9:

```
.\.venv\Scripts\python.exe `
learning/prompt-tuning-family/src/prefix_tuning_original_minimal.py
```

你应该看到类似信息:

```
kv_shape: (24, 2, 16, 5, 64)
saved_prefix_params: 245760
prefix_first_x_sees: {'P'}
infix_first_x_sees: {'x'}
attention: [('prefix_key', ...), ...]
```

如果能解释这些输出，说明你已经掌握论文的核心机制。

12. 最小代码片段

下面这个片段就是论文的机制缩影：

```
from prefix_tuning_original_minimal import PrefixConfig
from prefix_tuning_original_minimal import prefix_kv_shape
from prefix_tuning_original_minimal import saved_prefix_parameters

cfg = PrefixConfig(
    layers=24,
    hidden_dim=1024,
    num_heads=16,
    prefix_len=5,
)

print(prefix_kv_shape(cfg))
print(saved_prefix_parameters(cfg))
```

输出含义：

```
(24, 2, 16, 5, 64)
245760
```

解释：

- 24 层。
- 每层两个前缀：key 和 value。
- 16 个 attention heads。
- prefix 长度 5。
- 每个 head 的维度 64。
- 推理时每任务只存约 25 万个 prefix 参数。

再看 prefix vs infix：

```
from prefix_tuning_original_minimal import build_layout
from prefix_tuning_original_minimal import causal_reach

prefix_layout = build_layout("prefix", prefix_len=2, x_len=3, y_len=2)
infix_layout = build_layout("infix", prefix_len=2, x_len=3, y_len=2)

print(prefix_layout)
print(causal_reach(prefix_layout)[2])

print(infix_layout)
print(causal_reach(infix_layout)[0])
```

你要能说出:

- prefixing 让 x 的第一个 token 也能看到 prefix。
- infixing 不能影响 x 的表征。
- 这解释了 Table 4 中 infix 略弱。

13. 实验一: Table-to-text 主结果

论文第 6.1 节用 GPT-2 做 table-to-text。

数据集:

- E2E: restaurant domain, 约 50K examples。
- WebNLG: DBpedia triples, 训练 seen categories, 测试含 unseen categories。
- DART: open-domain table-to-text, 规模更大, 来源更多。

指标:

- BLEU。
- NIST。
- METEOR。
- ROUGE-L。
- CIDEr。
- TER。
- MoverScore。
- BERTScore。
- BLEURT。

不要试图背所有指标。重点是论文在比什么:

quality metric

vs

task-specific parameter count

主结论:

- Prefix-tuning 用约 0.1% 任务参数。
- 它优于同样 0.1% 参数量的 adapter。
- 它与 full fine-tuning 接近, 部分表格任务上更好。
- GPT-2 medium 和 GPT-2 large 都能成立。

Table 1 里的典型读法:

- E2E 上 GPT-2 medium prefix BLEU 约 69.7, fine-tuning 约 68.2。
- DART 上 prefix 和 fine-tuning 都在 46 BLEU 左右。
- WebNLG unseen categories 上 prefix 的优势很明显。

这支持论文的第一条证据:

prefix is expressive enough for structured generation,
while storing far fewer task-specific parameters.

14. 实验二: Summarization

论文第 6.2 节用 BART large 做 XSUM summarization。

结果没有 table-to-text 那么漂亮。

Table 2 大意:

- Full fine-tuning 仍然最好。
- Prefix 2% 参数略低于 full fine-tuning。
- Prefix 0.1% 参数更低一些。

典型数值:

- Fine-tuning ROUGE-L 约 37.25。
- Prefix 2% ROUGE-L 约 36.05。
- Prefix 0.1% ROUGE-L 约 35.05。

这说明:

Prefix-Tuning 不是在所有生成任务上都无损替代 full fine-tuning。

作者给出可能原因:

- XSUM 数据量更大。
- 输入文章比表格长很多。
- 摘要需要阅读理解和内容选择, 更复杂。

这是读论文时很重要的诚实点:

论文不是只报好消息。它承认 full-data summarization 上 prefix 有性能差距。

15. 实验三: Low-data setting

第 6.3 节是本篇最有启发的实验之一。

作者构造小数据集:

training examples = 50, 100, 200, 500

每个 size 采样多个数据集, 再用不同 random seeds, 减少偶然性。

Figure 3 显示:

- Low-data 下 prefix-tuning 通常优于 fine-tuning。
- table-to-text 平均有约 2.9 BLEU 优势。
- 训练数据越多, 差距越小。

为什么小数据更适合 prefix?

可以这样理解:

Full fine-tuning 的可动参数太多。小数据下, 它很容易把 pretrained LM 的通用能力破坏掉。

Prefix-Tuning 只调一小段 task vector。它更像给 frozen LM 找一个条件控制器, 不会大幅改写 LM 本身。

Figure 3 左侧 qualitative example 也很关键:

fine-tuning 在低数据下容易生成错误属性, 例如把 average customer rating 说成 low。prefix-tuning 仍可能 undergenerate, 但更少 hallucinate 表格里的属性。

这和今天我们理解 PEFT 的直觉一致:

小数据时, 限制可训练参数有时反而更泛化。

16. 实验四: Extrapolation

第 6.4 节考察训练主题和测试主题不同的情况。

WebNLG:

```
train: seen DBpedia categories
test:  unseen DBpedia categories
```

XSUM:

```
split 1:
  train news
  test sports
```

```
split 2:
  train some news subdomains
  test other news subdomains
```

Table 3 显示 XSUM 外推中 prefix 比 fine-tuning 好。例如 news-to-sports:

- Fine-tuning ROUGE-L 约 30.26。
- Prefix ROUGE-L 约 31.51。

within-news:

- Fine-tuning ROUGE-L 约 31.15。
- Prefix ROUGE-L 约 31.47。

WebNLG 的 unseen columns 也显示 prefix 有外推优势。

作者的解释是:

冻结 pretrained LM 参数可能保留了更好的通用语言和世界知识。Full fine-tuning 更新全部参数, 可能更容易贴合训练主题。

不过作者也诚实指出:

为什么 prefix 和 adapter 在外推上更好, 仍然是 open question。

这句话很适合记在笔记里。好论文不只是给答案, 也会指出证据还没有完全解释机制。

17. 实验五: Prefix length

Figure 4 研究 prefix length。

直觉上 prefix 越长, 可训练参数越多, 表达能力越强。

但实验不是单调无限上升。

论文观察:

- XSUM summarization 大约到 200 后收益到顶。
- DART table-to-text 大约到 10 后收益到顶。
- 再继续加长可能测试性能轻微下降。

作者解释:

更长 prefix 可能降低训练 loss, 但测试性能变差, 说明它可能开始过拟合。

工程上还有一个点:

较长 prefix 对推理速度影响不大，因为 GPU 上 prefix attention 可并行。但它仍然会增加 attention memory 和 KV cache 长度。

今天部署大模型时，这个点不能忽略。

18. 实验六: Embedding-only and infix ablation

Table 4 是本篇最关键的机制证据之一。

Embedding-only:

- 只学 virtual token 的 embedding。
- 上层 activation 由 Transformer 正常算。
- 可以看成 discrete prompt optimization 的连续上界。

结果:

- 明显弱于 full prefix-tuning。

这证明:

只在输入层放 soft prompt 不够。每层 K/V prefix 提供了更强的控制通道。

Infix-tuning:

- 把 trainable activation 放在 x 和 y 中间。
- 不能影响 x 的 activation。

结果:

- 稍弱于 prefix-tuning。

这证明:

prefix 放在序列开头有意义，因为它可以同时影响 input representation 和 output generation。

19. 实验七: Initialization

Figure 5 研究 prefix initialization。

论文发现:

- Random initialization 在低数据下表现差且方差大。
- 用真实词 activation 初始化明显更好。
- 与任务相关的词略好于无义词。
- 但真实词通常都比 random 好。

这背后的直觉:

如果 prefix 初始点已经位于 pretrained LM 熟悉的 activation manifold 附近，优化会更稳定。

这和 reparameterization 的作用一致:

二者都在帮助优化过程不要从完全无结构的自由向量开始乱跑。

20. Discussion: 个性化和批处理

论文第 8 节把 prefix-tuning 连接到更实际的场景。

20.1 Personalization

如果每个用户都有自己的数据，并且隐私要求用户数据隔离，那么可以给每个用户训练一个 prefix。

```
shared frozen LM
+ user A prefix
+ user B prefix
+ user C prefix
```

删除用户时，删除对应 prefix 即可。

这比为每个用户训练一整份 LM 更现实。

20.2 Batching across users

因为主干 LM 完全相同，不同用户的请求仍然可以放进同一个 batch。

只需要每条样本拼自己的 prefix。

Adapter-tuning 更难做到这一点，因为 adapter 插在层内部，不同用户使用不同 adapter 时 batch 计算更复杂。

20.3 Inductive bias

作者认为 prefix-tuning 的 inductive bias 是：

尽量保持 pretrained LM intact，只用 prefix 引导它。

这可能解释了外推能力。但论文也承认，这个解释还不是严格证明。

21. 与后续 PEFT 方法的关系

Prefix-Tuning 是 PEFT 家族里非常重要的一步。

它影响了后续几条路线：

Prompt Tuning:

- 更简单。
- 只在输入层学习 soft prompt。
- 后来的研究发现，大模型到 10B 以上时效果会显著改善。

P-Tuning:

- 也学 continuous prompt。
- 用 LSTM/MLP 做 reparameterization。
- 更偏 NLU。

P-Tuning v2:

- 回到每层 deep prompt。
- 去掉复杂 reparameterization。
- 面向 NLU 通用化。

LoRA:

- 不学 prefix。
- 学低秩权重更新。

- 后来成为大模型微调和部署中更主流的路线之一。

Adapter:

- 插入 task-specific module。
- 参数量通常比 prefix 多。
- 表达方式更像改变 hidden state residual。

今天看 Prefix-Tuning，它的意义不是“最流行的微调方法”，而是开辟了一个思想：

```
large frozen model
+ small task-specific control parameters
```

这就是现代 PEFT 的共同精神。

22. 常见误解

误解一：Prefix-Tuning 就是手写 prompt。

不对。手写 prompt 是离散文字。Prefix-Tuning 学的是连续 activation / key-value memory。

误解二：Prefix-Tuning 只是在输入前面加 embedding。

不对。论文的关键是每层 attention 的 prefix activation。Table 4 说明 embedding-only 明显更弱。

误解三：Prefix-Tuning 在所有任务上都等于 full fine-tuning。

不对。XSUM full-data summarization 上 prefix 低于 full fine-tuning。

误解四：Prefix 越长越好。

不对。Figure 4 显示有阈值，太长可能过拟合。

误解五：MLP reparameterization 推理时也必须保留。

不对。训练结束后可以丢弃 MLP，只保存最终 prefix。

误解六：Prefix-Tuning 没有推理成本。

不完全对。它减少了任务参数存储，但每层 attention 多了 prefix KV，KV cache 和 attention 长度仍会增加。

23. 30 到 60 分钟本地实验

建议实验顺序：

第一步，跑无依赖机制文件：

```
.\.venv\Scripts\python.exe `
learning\prompt-tuning-family\src\prefix_tuning_original_minimal.py
```

你要解释：

- kv_shape 每个维度是什么。
- saved_prefix_params 怎么算出来。
- 为什么 prefix first x sees 里有 P。
- 为什么 infix first x sees 里没有 P。
- attention 里 prefix key 为什么权重大。

第二步，跑对应测试：

```
.\.venv\Scripts\python.exe `
learning\prompt-tuning-family\src\tests\test_prefix_original_minimal.py
```

第三步，如果本机 HuggingFace 环境可用，再跑真实最小实现：

```
.\.venv\Scripts\python.exe `
learning\prompt-tuning-family\src\prefix_tuning_minimal.py
```

观察：

- trainable 参数量。
- logits shape。
- no reparam 消融的参数量。

第四步，做一个小改动：

在 prefix_tuning_original_minimal.py 里把：

```
prefix_len = 5
```

改成：

```
prefix_len = 10
```

重新计算：

- saved_prefix_params 是否翻倍。
- prefix_kv_shape 的第四维是否从 5 变成 10。

这个实验对应 Figure 4：

更长 prefix 表示更多可训练能力，但不保证测试集更好。

24. 用 AI agent 正确学习这篇

这篇很适合用 AI agent 训练你 “真的进脑子”。

不要让 agent 直接总结。应该让 agent 追问你。

可以这样提示：

我正在学习 Prefix-Tuning 论文。

请不要泛泛总结。

请按以下顺序考我：

1. 先让我解释 full fine-tuning 的存储痛点。
2. 再让我画 Figure 1 的 full fine-tuning vs prefix-tuning。
3. 再让我写出 prefix KV 的 shape。
4. 再让我解释 Equation 3。
5. 再让我解释 Table 4 为什么证明 embedding-only 不够。
6. 最后让我把本地 prefix_tuning_original_minimal.py 的输出逐行解释。

一次只问一个问题。

我回答后，请指出漏洞，并要求我用代码文件名或论文 figure/table 对齐。

学习时你要坚持三个动作。

动作一：先闭卷画图。

[prefix KV] + [normal KV] -> attention -> logits

画不出来，说明你只是记了名词。

动作二: 让 agent 逼你说 shape。

最低限度要会说:

```
prefix_kv = (layers, 2, heads, prefix_len, head_dim)
```

动作三: 让 agent 逼你联系证据。

例如:

为什么不是 embedding-only?

回答必须提 Table 4。

为什么 low-data 更好?

回答必须提 Figure 3 和过拟合直觉。

为什么外推更好?

回答必须提 Table 3 和 frozen LM 的归纳偏置。

好的 AI agent 学习不是让你更快跳过论文，而是让你更快暴露自己哪里没懂。

25. 局限性

局限一: Summarization full-data 上不完全追平。

XSUM 结果说明 prefix 不是无损替代。长输入、复杂内容选择、较大数据量都会让 full fine-tuning 仍有优势。

局限二: 超参敏感。

Prefix length、learning rate、初始化都会影响结果。论文第 7 节说明 random initialization 在 low-data 下方差大。

局限三: 机制解释还不完整。

论文看到 prefix 和 adapter 在 extrapolation 上好，但也承认原因仍是 open question。

局限四: prefix 不可解释。

它不是自然语言 token，很难直接读懂某个 prefix 学到了什么。

局限五: 推理 KV 成本仍在。

它节省任务参数存储，但 attention 需要处理额外 prefix positions。

局限六: 论文任务范围有限。

主要是 NLG:

- table-to-text。
- summarization。

对 NLU、代码、多模态、工具调用等后续场景，需要看后来的 Prompt Tuning、P-Tuning、P-Tuning v2、LoRA 等工作。

26. 现代意义

今天看 Prefix-Tuning，最重要的不是“我应该马上用它替代 LoRA”，而是理解它提出的工程范式：

```
freeze large pretrained model
learn a small task controller
share the backbone across tasks
```

这条范式后来贯穿：

- LoRA。
- Adapters。
- Soft prompts。
- Prompt pools。
- Multi-tenant LLM serving。
- Personalization。
- Private per-user adaptation。

同时它提醒我们：

参数高效不是只看 trainable parameter count。还要看：

- 是否能 batch。
- 是否增加 KV cache。
- 是否稳定。
- 是否容易初始化。
- 是否适合当前任务类型。
- 是否在 low-data 和 out-of-domain 上有证据。

27. 闭卷掌握检查

读完后，请闭卷回答。

1. Full fine-tuning 的部署痛点是什么？
2. Prefix-Tuning 和手写 prompt 的区别是什么？
3. Prefix-Tuning 和 input-level prompt tuning 的区别是什么？
4. Figure 1 中为什么 prefix-tuning 只需要保存小 prefix？
5. 对 autoregressive LM，[PREFIX; x; y] 中 prefix 如何影响 y？
6. 对 encoder-decoder LM，为什么 encoder 和 decoder 都可以加 prefix？
7. $\text{prefix_kv} = (L, 2, H, p, d_{\text{head}})$ 每一维是什么意思？
8. 为什么论文要用 MLP reparameterization？
9. 为什么训练结束后 MLP 可以丢弃？
10. Table 1 证明了什么？
11. Table 2 为什么是对论文结论的限制？
12. Figure 3 的 low-data 结果说明了什么？
13. Table 3 的 extrapolation 结果如何解释？
14. Figure 4 为什么说明 prefix length 不是越长越好？

15. Table 4 为什么是理解方法的关键证据?
16. Prefixing 为什么通常比 infixing 更合理?
17. Figure 5 对初始化有什么启发?
18. Prefix-Tuning 和 adapter-tuning 的部署差异是什么?
19. 本地 `prefix_tuning_original_minimal.py` 中哪个函数对应 prefix KV shape?
20. 如果有 100 个任务, full fine-tuning 和 prefix-tuning 的存储结构分别是什么?

28. 最小复述模板

闭卷时可以这样讲:

Prefix-Tuning 解决的是大语言模型下游生成任务 full fine-tuning 的存储和过拟合问题。它冻结 GPT-2 或 BART 的全部参数, 只训练一段 task-specific continuous prefix。这段 prefix 不是自然语言 prompt, 也不只是输入 embedding, 而是每层 attention 可见的 key/value activation。对 GPT-2, 可以看成序列 [PREFIX; x; y]; 对 BART, 可以在 encoder 和 decoder 两侧都加 prefix。训练目标仍然是最大化 $p(y \text{ given } x)$, 但 optimizer 只更新 prefix 参数。由于直接学完整 prefix 不稳定, 论文用 MLP reparameterization 生成 prefix, 训练后丢弃 MLP, 只保存最终 prefix。实验上, Table 1 显示它在 table-to-text 上用约 0.1% 任务参数接近或超过 full fine-tuning。Table 2 显示 XSUM full-data summarization 仍低于 full fine-tuning。Figure 3 和 Table 3 显示 low-data 与 extrapolation 更强。Table 4 说明 embedding-only 和 infix 不如每层 prefix。它的现代意义是: 一个 frozen backbone 加许多小 task controllers, 这正是后来 PEFT 和多租户大模型部署的重要思想。

如果你能跑通 `prefix_tuning_original_minimal.py`, 并解释 (24, 2, 16, 5, 64) 和 245760, 这篇就基本进脑子了。